ALUMNICONNECT

TESTING & EVALUATION PLAN

Integrity • Performance • Concurrency • Integration

A comprehensive verification strategy for the Alumni–Student Mentorship and Mock Interview Platform.

DOCUMENT CONTROL	VALUE
Project	AlumniConnect - Alumni–Student Mentorship Platform
Document	Testing and Evaluation Plan
Version / date	1.0 / 02 November 2026
Application baseline	Next.js + TypeScript; FastAPI modular monolith
Authoritative database	MySQL 8.x with InnoDB
Supporting stores	Redis cache / temporary state; Neo4j recommendation projection
Document status	Implementation-oriented test plan; recommended targets and decisions are labelled

TESTING PRINCIPLE

A feature is accepted only when its visible behaviour, authoritative database state, security boundary and asynchronous side effects agree. MySQL remains the source of truth; Redis and Neo4j are supporting stores and must be safe to lose, delay or rebuild.

Prepared by consolidating the supplied AlumniConnect blueprint and all previously provided design specifications.

Document map

SECTION	SUBJECT
01	Purpose, scope and test principles
02	System under test and quality objectives
03	Test strategy, levels and coverage
04	Environment, fixtures and data management
05	Requirements traceability and release priorities
06	Database integrity and relational validation
07	Workflow and end-to-end functional testing
08	Transaction, ACID and concurrency testing
09	Cross-database integration: Redis, Neo4j and outbox
10	Security, authorization and privacy verification
11	Performance and capacity evaluation
12	Reliability, resilience and failure injection
13	Reporting, query plans and operational observability
14	Execution process, defect management and exit gates
15	Acceptance demonstration and evidence pack
16	Open decisions, assumptions and source traceability

PROVENANCE LABELS

[B] Blueprint-defined; [P] requirement stated in the supplied PRD; [C] course concept demonstrated from DBMS materials; [R] recommended test/engineering practice added to make evaluation repeatable. Proposed performance targets are not represented as guaranteed requirements.

How to use this plan

Use the plan as the master test specification. Keep each test ID stable in the test tracker, automation report and demonstration evidence. Map any final schema renaming to the logical entities in the ER model rather than changing the business meaning of a test.

Execute the core relational and authorization tests before enabling Redis and Neo4j. Then run integration tests that exercise the outbox and secondary-store projections. Run performance tests only after correctness and data fixtures are stable.

1. Purpose, scope and test principles

This document defines how AlumniConnect will be verified before the DBMS project is accepted. It combines functional, database, concurrency, security, cross-store, performance and recovery tests into one plan. The objective is not merely to prove that the interface renders or that individual API endpoints return 200; it is to prove that the platform preserves the business rules and database consistency established in the supplied design documents.

1.1 Test objectives

- Verify referential integrity, uniqueness, domain constraints, lifecycle states and historical-record retention in MySQL.
- Prove ACID transaction boundaries for request acceptance, booking, cancellation, session outcome, feedback/review and outbox event creation.
- Prove that simultaneous operations cannot exceed mentor capacity, allocate one request twice, double-book a slot or create overlapping participant bookings.
- Verify that role-based permissions are combined with record ownership, assignment and participant checks at the FastAPI boundary.
- Verify cache-aside behaviour, TTL expiry and outbox-driven Redis invalidation without allowing Redis to become business authority.
- Verify Neo4j node/relationship identity, traversal output, projection idempotency, source-version ordering, rebuild and MySQL eligibility filtering.
- Measure response latency, throughput, errors, database/graph/cache behaviour and background-worker lag using repeatable workload and dataset conditions.
- Prove graceful recovery when Redis, Neo4j, the notification provider or the outbox worker is unavailable.

1.2 Scope included in the core release

AREA	IN-SCOPE VERIFICATION
Identity and access	Registration/sign-in, role assignment, account status, role/profile linking, password handling and access denial.
Profiles and catalogues	Student/mentor profiles, department, company, skills, domains, many-to-many links and public/private projections.
Mentor discovery	SQL filters, Neo4j traversal, candidate explanation, MySQL eligibility/capacity filtering and optional Redis result cache.
Requests and allocation	Submit, accept, reject, pending cancellation, capacity checks, unique allocation and history.
Availability and booking	Slot publication, overlap validation, duplicate retry, booking confirmation/cancellation and slot state.
Sessions and feedback	Scheduled/completed/cancelled/no-show outcome; participant feedback; completed-session mentor review.
Notifications and operations	Transactional outbox, notification recipients, worker retries, audit records and admin reports.
Supporting stores	Redis cache/temporary state and Neo4j relationship projection.

1.3 Explicitly out of scope for core acceptance

Video-conferencing infrastructure, real-money payments, independent microservice deployment and Redis Stack vector/semantic discovery are outside the core release described by the PRD. Vector search may be tested as a Phase 2 extension only after the SQL and graph recommendation path passes.

System boundary: the project uses a modular monolith. The browser calls FastAPI; FastAPI coordinates MySQL, Redis and Neo4j. MySQL is authoritative for identities, requests, allocations, availability, bookings, sessions, feedback, reviews, notifications and audit records.

2. System under test and quality objectives

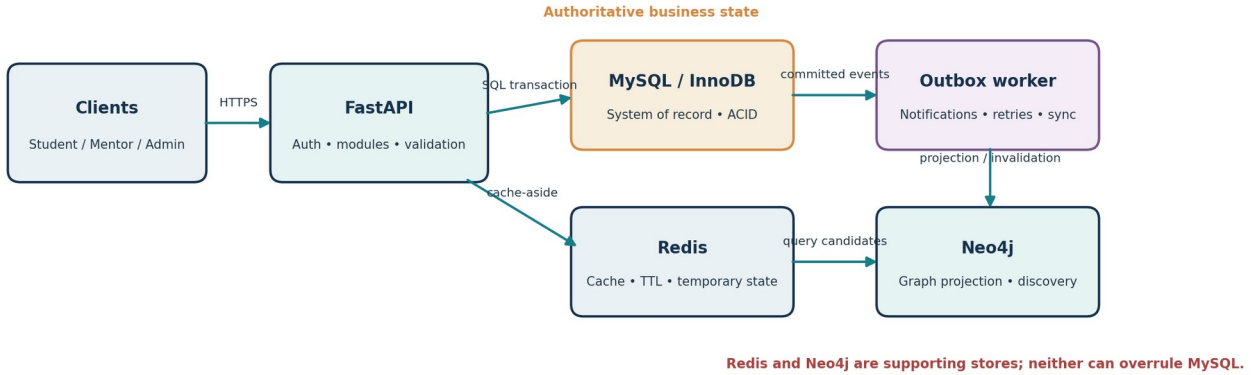


Figure 1. Components and consistency boundaries that must be exercised by the test plan.

2.1 System responsibilities

COMPONENT	TESTED RESPONSIBILITY	MUST NOT BE TREATED AS
Next.js clients	Role-specific forms, navigation, workflow displays, API integration and client-side feedback.	An authorization boundary; hidden buttons do not secure APIs.
FastAPI modular monolith	Authentication, authorization, validation, business modules, transaction orchestration and data adapters.	A trust boundary for user-supplied actor IDs or roles.
MySQL / InnoDB	Durable business records, constraints, procedures, transactions, locks, audit and outbox.	A cache or eventual projection.
Redis	Cache-aside profile/catalogue/recommendation values and short-lived operational state.	Authority for slot availability, allocation status, user authorization or durable history.
Neo4j	Graph projection for skill/domain/company/department paths and explainable recommendation candidates.	Authority for user status, mentor capacity, availability or booking.
Outbox worker	Post-commit notifications, graph projection updates and Redis invalidation with retries.	A source of events not committed by MySQL.

2.2 Quality attributes

QUALITY ATTRIBUTE	WHAT MUST BE PROVEN	EVIDENCE
Correctness	Returned API/UI state reflects valid MySQL records and permitted transitions.	API response + before/after SQL state.
Integrity	No orphan, duplicate, invalid-state or historically destructive data.	Constraint tests, SQL assertions and audit rows.
Isolation / concurrency	Concurrent requests cannot violate slot, overlap or capacity rules.	Two-connection race transcripts and final row counts.
Security / privacy	Only authorized users access records and only approved fields are exposed.	Positive/negative role matrix and response/log inspection.

QUALITY ATTRIBUTE	WHAT MUST BE PROVEN	EVIDENCE
Cross-store consistency	Redis/Neo4j converge from committed events and cannot overrule MySQL.	Outbox status, cache invalidation, graph sync and replay evidence.
Performance	Common reads, recommendation and booking meet agreed initial project targets.	Repeatable load report with environment and dataset.
Reliability	A secondary-store/provider failure does not roll back an already committed business action.	Failure-injection logs and successful recovery.
Usability / accessibility	Representative journeys are understandable, responsive and keyboard-operable.	Manual review checklist and screenshots.

3. Test strategy, levels and coverage

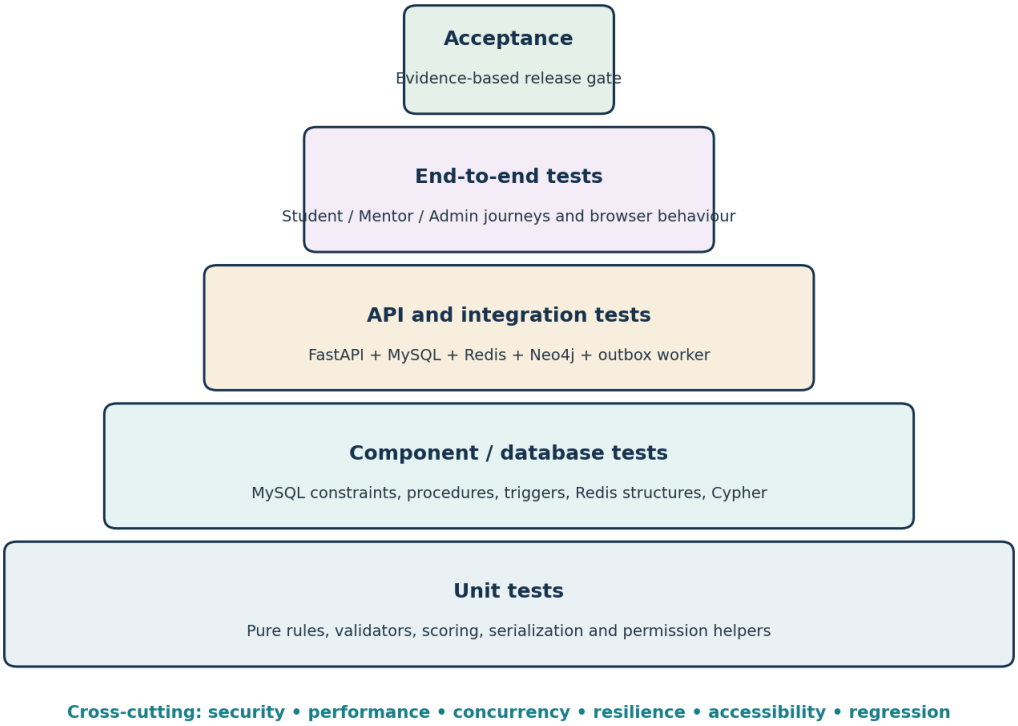


Figure 2. A layered strategy: fast unit tests support deeper database, integration and end-to-end verification.

3.1 Test levels

LEVEL	TARGET	EXAMPLES / EXIT CONDITION
Unit	Pure business helpers and utilities.	Status transition validators, overlap predicate, key builders, permission evaluators, ranking explanation format; deterministic tests pass.
Schema / database component	MySQL objects and constraints.	DDL, PK/FK/UNIQUE/CHECK, views, triggers, stored procedures and rollback; database invariants pass.
Supporting-store component	Redis and Neo4j in isolation.	Redis data structures/TTL; Neo4j uniqueness constraints, MERGE idempotency and Cypher traversal.
API / module integration	FastAPI modules and repositories.	Identity/profile, request/allocation, booking/session/feedback and admin endpoints.
Cross-store integration	Outbox worker and secondary stores.	Committed event to Redis invalidation / Neo4j update / notification delivery; retries do not duplicate effects.
System / end-to-end	Complete role-based user journeys.	Student discovery through review; mentor inbox/session; admin catalogue/report workflow.
Non-functional	Performance, resilience, security and usability.	50-user baseline load, fault injection, access matrix, responsive/accessibility checks.
Acceptance / demonstration	Course and product acceptance evidence.	Live SQL, ACID, concurrency, Redis, Neo4j, security and analytics evidence.

3.2 Test types and required coverage

- Positive tests: valid actors, valid state, complete profile, eligible allocation and available slot.

- Negative tests: wrong role, wrong owner, inactive account, invalid state, duplicate IDs, missing references and prohibited transition.
- Boundary tests: time interval edges, capacity exactly at limit, empty profile values, rating bounds and TTL expiry boundary.
- Concurrency tests: same resource claimed simultaneously using two or more independent database connections.
- Failure tests: injected exception between writes, worker crash, Redis/Neo4j outage and provider timeout.
- Regression tests: rerun after schema migration, trigger/procedure change, cache-key change, role rule change or graph projection update.

3.3 Test priority

PRIORITY	MEANING	RELEASE RULE
P0 - Critical	Could create invalid booking/allocation, expose another person's data, bypass authorization, lose committed business state or corrupt history.	Must pass; no waiver without explicit project-owner decision and documented compensating control.
P1 - High	Core workflow or cross-store integration failure; recommendations, cache invalidation, worker retry, admin reporting.	Must pass for the affected module/release scope.
P2 - Medium	Usability, non-critical analytics, optional display/feed/ranking edge cases.	May be accepted with owner, workaround and due date.
P3 - Low	Cosmetic or non-blocking display issue.	Track and schedule; must not conceal a correctness defect.

4. Environment, fixtures and data management

4.1 Test environments

ENVIRONMENT	PURPOSE	REQUIREMENTS
Local developer	Fast unit and component feedback.	Isolated MySQL schema, Redis key prefix and Neo4j test database; synthetic data only.
Integration / CI	Repeatable API, database and worker tests.	Pinned service images/versions; migration from empty schema; deterministic fixtures; clean keyspace per run.
Staging / demo	Acceptance and performance evidence.	Production-like configuration, test accounts, observability enabled, no real personal data.
Performance sandbox	Load, contention and soak tests.	Dedicated resources; fixed dataset; no unrelated workloads; repeatable run scripts.

4.2 Minimum test data entities

FIXTURE GROUP	REQUIRED CASES
Users and profiles	At least one active Student, two active Mentors, one Administrator, a suspended account, a user with both profile types only if that policy is approved, and a user with no role-specific profile.
Reference data	Multiple departments, companies, skills and mentorship domains; active and retired catalogue records.
Skills / domains	Students with overlapping and non-overlapping skills/interests; mentors with shared and different expertise.
Requests / allocations	PENDING, ACCEPTED, REJECTED and CANCELLED requests; active allocation; full-capacity mentor; ended allocation.
Availability / bookings	OPEN, BOOKED and BLOCKED slots; adjacent slots; overlapping slots; past slot; future slot; cancelled booking.
Sessions / feedback	SCHEDULED, COMPLETED, CANCELLED and NO_SHOW; participant feedback; valid/invalid review fixtures.
Operations	Pending, processed and failed outbox events; duplicate event ID; event out of order; failed notification delivery.

4.3 Data management rules

- Use synthetic accounts and profile text; do not seed production email addresses, phone numbers, tokens, password hashes or private notes.
- Every test should declare preconditions and either create an isolated fixture or reset the required records before execution.
- Use stable numeric IDs or deterministic UUIDs in tests so SQL, Redis keys, Neo4j nodes and logs can be correlated.
- Keep separate Redis key namespaces and Neo4j databases for dev, CI and staging; tests must not flush a shared environment.
- Freeze or record the timezone, server clock assumption, database isolation level, schema version, Redis configuration and graph index state for reproducibility.
- Preserve failure evidence before cleanup: transaction logs, SQL row snapshots, outbox rows, Redis TTL/key state, Neo4j results and worker logs.

5. Requirements traceability and release priorities

SOURCE REQUIREMENT / DESIGN	TEST GROUP	RELEASE EVIDENCE
Conceptual ER Model: 24 entities, 36 relationships and cardinalities.	DBI relational integrity and migration tests.	PK/FK/UNIQUE/CHECK results; cardinality queries.
PRD BR-01..BR-12: identity, M:N skill links, request/allocation, booking/session distinction, slot uniqueness, feedback/review and source-of-truth.	DBI + WF + TX.	State tests and SQL snapshots.
PRD FR-20 / FR-23: outbox, Redis invalidation, cache-aside, TTL and retries.	OUT + RED.	Commit/rollback, cache hit/miss/expiry and replay reports.
PRD NFR-01 / NFR-02: server-side authorization and privacy.	SEC.	Negative API matrix and public-field inspection.
PRD NFR-03 / NFR-04: transactional integrity and cross-store consistency.	TX + OUT + NEO + RED.	Concurrent same-slot test, failure recovery and projection convergence.
PRD NFR-05: proposed initial performance baseline.	PERF.	p50/p95 latency, throughput, errors, environment and dataset.
PRD NFR-07 / NFR-08: failure recovery and auditability.	RES + AUD.	Failure injection and immutable history checks.
Workflow §19: WF-01..WF-16 lifecycle and transaction checks.	WF / TX.	All applicable cases pass with independent SQL sessions for races.
Business Rules Catalogue §16: BR-T01..BR-T34.	DBI / WF / TX / SEC.	Business-rule catalogue mapped to automated test IDs.
Redis Design §18: RED-01..RED-16.	RED.	Cache CLI and API behaviour evidence.
Neo4j Graph Design §17: G-01..G-12.	NEO.	Cypher traversal, replay, version and rebuild evidence.
Transaction Design §22-23: TX-01..TX-14 and failure injection.	TX / RES / OUT.	Two-connection transcripts and rollback snapshots.
Security Design §13: SEC-01..SEC-24.	SEC.	Role/ownership tests and sensitive data/log review.

5.1 Test ID convention

The test identifiers below use stable prefixes: DBI (database integrity), WF (workflow), TX (transaction/concurrency), OUT (outbox), RED (Redis), NEO (Neo4j), SEC (security), PERF (performance), RES (resilience), AUD (audit) and UX (usability). Existing IDs from the earlier design documents are retained where possible, including WF-01..WF-16, TX-01..TX-14, RED-01..RED-16, G-01..G-12 and SEC-01..SEC-24.

6. Database integrity and relational validation

Run these tests against the migrated MySQL schema, not only against mocked repositories. The final physical schema may use different table/column names; map each test to the final implementation while preserving the logical rule.

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
DBI-01	P0	Duplicate primary identifier is inserted into User, Student Profile or Mentor Profile.	Insert one row, then repeat the same PK.	Second write fails; one authoritative row remains. Capture constraint name.
DBI-02	P0	A profile, request, booking or feedback row references a nonexistent parent.	Attempt insert with invalid FK.	FK rejects the row; no orphan data survives.
DBI-03	P0	A Student is linked to the same Skill twice.	Insert duplicate (student_id, skill_id).	Composite uniqueness rejects duplicate association. Repeat for Mentor Skill and domain junctions.
DBI-04	P0	A request is inserted with an unknown status or invalid required fields.	Use unsupported enum/domain value or NULL for mandatory fields.	CHECK/NOT NULL/domain guard rejects invalid state.
DBI-05	P0	Booking/slot has end_at <= start_at.	Insert zero-duration or negative-duration interval.	CHECK/procedure rejects; no invalid appointment persists.
DBI-06	P0	One request is associated with two allocations.	Attempt second allocation for same request.	UNIQUE(mentorship_request_id) or equivalent rejects second allocation.
DBI-07	P0	One availability slot has two active confirmed bookings.	Attempt second active booking for same availability_id.	Unique active slot rule/slot lock rejects second claim.
DBI-08	P0	Two role-specific profiles accidentally duplicate a shared account identity.	Insert a second Student Profile for same user; repeat Mentor Profile.	Unique user_id per profile extension preserves 1:0..1 cardinality.
DBI-09	P1	Booking-to-session cardinality permits more than one session for one booking.	Attempt duplicate session creation.	UNIQUE(booking_id) or equivalent permits at most one session per booking.
DBI-10	P1	Mentor Review is duplicated for one student and one completed session.	Insert the same review key twice.	Unique(session_id, student_id) or approved equivalent blocks duplicate; original history remains.
DBI-11	P0	Account is deactivated while historical requests, allocations, bookings or reviews exist.	Soft-deactivate account; query related historical records.	Historical rows remain; no cascade-delete of business history.
DBI-12	P1	A retired Skill/Domain remains referenced by historical profile or request data.	Retire catalogue entry and query links/history.	Reference remains valid or is explicitly inactive; no silent destructive cascade.
DBI-13	P1	Invalid session or request state transition is attempted.	Try ACCEPTED -> PENDING or COMPLETED -> SCHEDULED.	Procedure/trigger rejects; audit/history has no false transition.
DBI-14	P1	Audit trigger records a state change.	Update request/session status through valid path.	Exactly one audit event records actor, time, target and old/new state; rollback also rolls audit back.

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
DBI-15	P1	Public mentor view includes private account fields.	Inspect v_public_mentor_directory/API projection columns.	Private email/phone, auth fields, admin notes and participant-only notes are absent.
DBI-16	P1	Indexes support high-selectivity access paths.	EXPLAIN mentor directory, booking calendar, request inbox and session history.	Plans use intended indexes where appropriate; record plan, rows examined and changes.

6.1 Domain invariants to assert after test runs

- Each User has no more than one Student Profile and no more than one Mentor Profile, subject to the approved role policy.
- Every request identifies one student, one mentor and a valid mentorship domain/objective; request status is one of PENDING, ACCEPTED, REJECTED or CANCELLED.
- At most one Mentor Allocation is created from a request; active mentor capacity is not exceeded.
- Every confirmed Booking references the correct student, mentor, availability slot and valid active allocation where that booking type requires it.
- No active booking duplicates one slot; no confirmed appointments overlap for the same student or mentor under the half-open interval rule.
- Every Mentorship Session references a valid Booking; feedback is by actual participants after the session has taken place; mentor review requires a completed session.
- Notification/outbox/audit records are tied to a committed business action and preserve required history.

7. Workflow and end-to-end functional testing

7.1 Identity, profiles and catalogue workflow

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
WF-ID-01	P0	Active Student signs in and opens own dashboard.	Authenticate; GET profile/dashboard.	Correct Student profile and own data returned; no cross-user fields.
WF-ID-02	P0	Active Mentor signs in and edits own professional profile.	Update headline, company, experience, skills/domains.	MySQL reflects update; audit/outbox emitted where required; profile/recommendation cache invalidated after commit.
WF-ID-03	P1	Administrator retires a skill/domain.	Update catalogue status and view dependent profiles.	Existing history retained; new selection policy follows configured active/inactive rule.
WF-ID-04	P0	Suspended account attempts a write action.	Attempt request, accept, publish slot, book or review.	Write denied; historical read remains according to policy.
WF-CAT-01	P1	Student links multiple skills/interests and Mentor links multiple skills/expertise domains.	Create and remove associations.	M:N relations persist without duplication; retrieval and graph projection agree after sync.

7.2 Request and allocation lifecycle

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
WF-01	P0	Student submits valid request to active/discoverable mentor.	POST request with authenticated Student principal.	One PENDING request; one REQUEST_SUBMITTED outbox event to target Mentor.
WF-02	P0	Target Mentor accepts PENDING request with capacity.	Accept through API/procedure.	Request ACCEPTED; one ACTIVE allocation; one outbox event; all writes commit together.
WF-03	P0	Mentor attempts to accept an already REJECTED/CANCELLED request.	Submit decision after terminal state.	Operation rejected; no allocation or duplicate notification.
WF-04	P0	Two accepts compete for last mentor capacity.	Two independent connections synchronize acceptance.	Only one permitted allocation commits; loser re-checks capacity and fails cleanly.
WF-05	P1	Same acceptance request is retried after timeout.	Repeat identical idempotency key/fingerprint.	Same result or deterministic conflict; no duplicate allocation.
WF-06	P1	Mentor rejects request.	Reject a PENDING request with valid actor.	Request REJECTED; student notification intent committed; request row retained.
WF-07	P1	Student cancels own PENDING request.	Cancel as owner, then query history.	Request CANCELLED; history retained; notification follows configured policy.

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
WF-08	P0	A different Student attempts to cancel someone else's request.	Change request_id in URL/body.	Denied; status and outbox unchanged.

7.3 Availability and booking workflow

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
WF-AVL-01	P1	Active Mentor publishes valid future OPEN slot.	Create availability slot with start_at < end_at.	Slot persisted and visible in mentor calendar.
WF-AVL-02	P0	Inactive/suspended Mentor publishes availability.	Attempt to create slot.	Rejected; no slot inserted.
WF-AVL-03	P1	Mentor attempts overlapping active slots.	Publish overlapping interval.	Rejected unless group/parallel scheduling is explicitly enabled.
WF-06	P0	Two students book same slot simultaneously.	Two DB connections + barrier; same slot ID.	Exactly one CONFIRMED booking; slot BOOKED once; loser gets conflict/unavailable.
WF-07	P0	One student books two overlapping slots with different mentors concurrently.	Two connections; overlapping time intervals.	At most one confirmed appointment survives.
WF-08	P0	Mentor has an overlapping confirmed appointment.	New student attempts conflicting booking.	New booking rejected; no booking/session/outbox partial write.
WF-09	P0	Outbox insert fails after booking writes.	Inject failure before outbox insertion.	Entire booking transaction rolls back; slot remains OPEN/AVAILABLE.
WF-10	P1	Confirmed booking is cancelled within configured cutoff.	Cancel as an authorized participant.	Booking retained as CANCELLED; slot release follows explicit policy; both participants notified.
WF-BOOK-11	P1	Slot is displayed OPEN in Redis but MySQL slot has already been claimed.	Stale availability preview then attempt booking.	MySQL locking validation rejects; stale preview cannot approve booking.

7.4 Session, feedback and review workflow

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
WF-11	P1	Assigned Mentor marks SCHEDULED session COMPLETED and records outcome.	Write topics, skills assessed, observations, improvements and next steps.	Outcome and notes persist; session history retained; correct outbox event committed.
WF-12	P0	Terminal COMPLETED session is changed back to SCHEDULED.	Attempt invalid transition.	Trigger/procedure rejects; no state regression.
WF-13	P0	Nonparticipant submits session feedback.	Use unrelated Student/Mentor principal.	Denied; no feedback row or notification event.
WF-14	P0	Student submits a second mentor review for same completed session.	Repeat review with same session/student.	Duplicate rejected; original review preserved.

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
WF-15	P1	Notification provider temporarily fails after feedback event committed.	Simulate provider outage.	Business data remains committed; event pending/retryable; no duplicate visible notification.
WF-SES-16	P0	Review submitted for CANCELLED or NO_SHOW session.	Insert review via API and direct service path.	Rejected; no review or event created.

7.5 Administrator and analytics workflow

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
WF-ADM-01	P0	Administrator updates role/status/reference catalogue.	Perform permitted admin change.	MySQL state updated; significant change audited; permissions reload from authoritative state.
WF-ADM-02	P0	Student/Mentor attempts catalogue or role management.	Call admin API directly.	Denied; no protected data or change.
WF-ADM-03	P1	Administrator views request-to-allocation-to-session funnel.	Run report on known fixture population.	Counts match SQL base records and filters; read-only operation.
WF-ADM-04	P1	Administrator views mentor rating summary.	Compare AVG/COUNT from reviews and completed sessions.	Aggregate is reproducible and excludes invalid/non-completed session reviews.
WF-ADM-05	P1	Catalogue item is retired while old history references it.	Retire item and query report/history.	Existing records remain readable; no cascade deletion.

8. Transaction, ACID and concurrency testing

8.1 ACID acceptance criteria

PROPERTY	ALUMNICONNECT ASSERTION	TEST EVIDENCE
Atomicity	All writes in a business operation succeed together or all roll back. Booking includes the booking, slot state and outbox intent; acceptance includes request state, allocation and outbox.	Injected failure at each write boundary; SQL snapshot after rollback.
Consistency	Every committed transaction preserves constraints and business invariants.	Post-transaction integrity queries; constraint failures.
Isolation	Concurrent operations cannot produce invalid intermediate/final states.	Two independent connections, lock waits, final counts and statuses.
Durability	Committed business rows and outbox intents survive database restart according to configured durability/backup settings.	Commit, controlled restart/recovery and post-restart SQL verification.

8.2 Concurrency test execution method

1. Use at least two independent MySQL connections; do not simulate a race with sequential calls in one session.
2. Prepare one immutable test fixture per race (request, slot, allocation, student and mentor) and verify its preconditions before the race.
3. Use a synchronization barrier so both transactions reach the contested operation at nearly the same time.
4. Capture transaction start/end times, connection IDs, lock wait, SQL error, API response and relevant rows after both operations finish.
5. Repeat each scenario multiple times under the configured InnoDB isolation level and final indexes; record test count and observed distribution.
6. Assert the final invariant, not only the HTTP result: exact counts/statuses, no duplicate outbox, and no orphan session.

LOCKING BASELINE

Use SELECT ... FOR UPDATE for mutable/contested rows, a consistent lock order such as student profile → mentor profile → allocation → availability slot → booking/session, and short transactions. Do not call Redis, Neo4j or external notification providers while holding SQL locks.

8.3 Concurrency catalogue

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
TX-01	P0	Two actors accept the same request simultaneously.	Separate connections; same request; barrier.	One allocation only; request ACCEPTED once; loser receives already-processed/conflict.
TX-02	P0	Two acceptances for different requests compete for mentor's last capacity.	Separate requests; mentor capacity set to one remaining.	Active allocation count never exceeds configured capacity.
TX-03	P0	Two students book the same slot simultaneously.	Two student principals; same availability ID.	Exactly one CONFIRMED booking; slot BOOKED.
TX-04	P0	One student attempts overlapping slots with different mentors.	Two availability IDs; overlapping intervals.	At most one confirmed booking; student guard row serializes check.
TX-05	P0	Two students attempt overlapping slots with same mentor.	Same mentor, different slots, conflicting intervals.	Second transaction rejected after mentor guard/check.

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
TX-06	P1	Adjacent appointments [10:00,10:30) and [10:30,11:00).	Create adjacent time intervals.	Both may commit; half-open interval predicate treats them as non-overlapping.
TX-07	P0	Booking API retry uses same idempotency key and fingerprint.	Repeat same request after simulated timeout.	Original booking result returned; no second booking/session.
TX-08	P0	Same idempotency key is reused with different payload.	Change requested slot or body while reusing key.	Conflict rejected; no second side effect.
TX-09	P0	Failure after booking insert but before slot update.	Inject exception in transaction.	Full rollback; no booking/session/outbox; slot stays OPEN.
TX-10	P0	Outbox worker is offline during booking commit.	Stop worker; complete booking.	Booking commits; outbox stays PENDING; later worker delivers event.
TX-11	P1	Worker crashes after provider accepts notification but before SENT update.	Inject crash at post-provider/pre-update boundary.	Retry occurs; event/recipient idempotency suppresses duplicate visible delivery where provider supports it.
TX-12	P1	Deadlock error 1213 is induced.	Run opposing lock acquisition pattern in controlled test.	Victim transaction fully rolls back; whole operation retried with bounded retry/jitter.
TX-13	P0	Feedback submitted while session not COMPLETED.	Attempt as participant.	Rejected; no feedback/outbox.
TX-14	P0	Two mentor reviews are submitted for one session by same Student.	Two independent calls or DB sessions.	One accepted; duplicate blocked by unique key.

8.4 Isolation anomaly verification

ANOMALY	ALUMNICONNECT SCENARIO	CONTROL / ASSERTION
Dirty read	Read slot or allocation changed in a transaction that later rolls back.	Business reads must not use READ UNCOMMITTED; rolled-back state is never exposed as committed.
Non-repeatable read	Allocation read ACTIVE, then another transaction ends it during decision.	Use an appropriate transaction snapshot or locking read when a stable eligibility decision is needed; recheck after lock.
Lost update	Two updates to the same slot or booking state.	Row lock, state recheck and controlled procedure prevent overwrite.
Phantom / overlap race	Two transactions both query zero overlapping bookings and insert different overlapping slots.	Lock participant guard rows in fixed order; all booking writers follow same discipline; supporting indexes present.
Duplicate submission	Browser retries booking after timeout.	Durable MySQL uniqueness/idempotency protection; return original result.

8.5 Failure injection points

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
FI-01	P0	After request update, before allocation insert.	Throw exception.	Request remains PENDING; no allocation or acceptance outbox event.

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
FI-02	P0	After allocation insert, before request status update.	Throw exception.	Allocation is rolled back; request remains PENDING.
FI-03	P0	After booking insert, before slot update.	Throw exception.	No booking/session/outbox survives; slot remains OPEN.
FI-04	P0	After slot update, before outbox insert.	Force outbox constraint/insert failure.	Slot and booking changes roll back.
FI-05	P1	After SQL commit, Redis is unavailable.	Stop Redis during worker processing.	Business state remains committed; outbox pending/retryable; cache rebuilt later.
FI-06	P1	After SQL commit, Neo4j is unavailable.	Stop Neo4j.	Business state remains committed; graph event retried; MySQL decision path remains correct.
FI-07	P1	After provider sends notification, worker crashes.	Crash before marking sent.	Retry dedupes delivery by event/recipient idempotency.

9. Cross-database integration: Redis, Neo4j and outbox

9.1 Transactional outbox tests

The transactional outbox is the integration boundary between MySQL and the secondary effects. Verify that business data and outbox intent commit together, while notification delivery, Neo4j projection and Redis invalidation occur asynchronously after commit.

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
OUT-01	P0	Business transaction commits with outbox intent.	Accept request or book slot.	Business row and matching event both committed; event references stable aggregate ID and event ID.
OUT-02	P0	Business transaction rolls back.	Inject failure before COMMIT.	Neither business state nor outbox event is visible to worker.
OUT-03	P1	Worker retries a transient failure.	Make provider/Redis/Neo4j fail once then recover.	Pending event is retried; eventual completion recorded.
OUT-04	P1	Duplicate event is delivered twice.	Replay same event ID.	No duplicate notification, duplicate graph relationships or unsafe repeated cache epoch increment.
OUT-05	P1	Unknown/malformed event type arrives.	Send invalid event payload.	Worker quarantines/rejects; no arbitrary write or unauthorized delivery.
OUT-06	P1	Outbox delivery repeatedly fails.	Exceed configured retry policy.	Event remains diagnosable in failed/dead-letter state; business transaction is not undone.
OUT-07	P1	Two workers claim the same event concurrently.	Run competing worker processes.	Claim/lease/locking strategy prevents duplicate concurrent processing or dedupes all side effects.

9.2 Redis integration catalogue

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
RED-01	P1	First mentor-card request with empty cache.	Delete key; call public mentor-card API.	API loads public-safe projection from MySQL and writes HASH with TTL.
RED-02	P1	Repeat same mentor-card request.	Call same endpoint with authorized actor.	Cache hit returns same public projection; authorization still executes.
RED-03	P1	Profile cache TTL expires.	Wait until TTL or set tiny TTL in test.	Key disappears; next request rebuilds from MySQL.
RED-04	P1	Mentor headline/experience changes.	Commit profile update and process outbox.	Mentor profile key invalidated; next read is fresh.

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
RED-05	P1	Mentor skills/domains change.	Update skills/domain links in MySQL.	Profile/recommendation cache invalidated; graph sync may be asynchronous.
RED-06	P1	Admin updates skill/domain catalogue.	Change catalogue and process event.	Catalogue key deleted/versioned; next read reloads current values.
RED-07	P1	Recommendation cached before mentor eligibility changes.	Update mentor status/capacity/skills; process invalidation.	Old recommendation epoch not reused; MySQL revalidates current eligibility.
RED-08	P0	Unauthorized actor requests cached mentor card.	Prime key with authorized request; call with unauthorized principal.	API denies; cache hit does not bypass permission.
RED-09	P0	Inspect public mentor cache payload.	HGETALL/inspect serialized response.	No private email/phone, token, password hash, internal notes or admin-only fields.
RED-10	P1	Redis unavailable during profile read.	Stop Redis.	Application falls back to MySQL or controlled failure; no business-state corruption.
RED-11	P1	Outbox retries the same invalidation event.	Replay event ID.	Duplicate event safely suppressed; recommendation epoch not incremented without bound.
RED-12	P0	Redis preview says OPEN while MySQL slot is claimed.	Stale cache; attempt booking.	Exactly one MySQL booking; losing attempt gets conflict/unavailable.
RED-13	P1	Rate-limit counter increments in one window.	Send calls in same key bucket.	Counter increments atomically and expires after configured window.
RED-14	P0	Idempotency key retried with same payload.	Repeat request with same key and fingerprint.	Original outcome returned; no duplicate booking/allocation.
RED-15	P0	Idempotency key reused with different payload.	Reuse key with modified body.	Request rejected as fingerprint conflict.
RED-16	P2	Recent activity feed receives many events.	Append activity repeatedly.	Feed length and TTL remain bounded; MySQL retains complete history.

9.3 Neo4j graph integration catalogue

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
G-01	P1	Create User with Student profile projection.	Process identity/profile event.	Exactly one User node, one Student node and one HAS_STUDENT_PROFILE relationship.
G-02	P1	Same account also has Mentor profile.	Project second profile for same source user.	Both profile nodes point to same User node; no duplicate User node.
G-03	P1	Student and Mentor share Python skill.	Load fixture and run skill traversal.	Candidate returns Mentor with Python in sharedSkills evidence.

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
G-04	P1	Student interested in MLOps; Mentor specialises in MLOps.	Run domain traversal.	Candidate returned with domain relevance evidence.
G-05	P2	Mentor belongs to student's department.	Run department alumni traversal.	Candidate returned with shared department evidence.
G-06	P1	Same outbox event processed twice.	Replay event.	No duplicate nodes or logical relationships; graph state remains correct.
G-07	P1	Older sourceVersion arrives after newer event.	Process version 42 then version 41.	Older event ignored; newer projection remains.
G-08	P0	Mentor is deactivated in MySQL and projected.	Change account state and sync.	Graph query excludes inactive mentor; MySQL eligibility also rechecks.
G-09	P0	Graph recommends mentor at full capacity.	Return candidate then run MySQL eligibility check.	MySQL excludes candidate from actionable result.
G-10	P1	Neo4j unavailable during profile update.	Stop Neo4j; commit MySQL profile update.	MySQL commits; outbox remains retryable; graph catches up after recovery.
G-11	P1	Full graph projection rebuild.	Rebuild from MySQL authoritative rows/events.	Nodes and edges reconstructed without losing business data; deterministic counts.
G-12	P1	Recommendation response contains explanation.	Run multi-factor matching.	Every displayed reason is supported by graph evidence or SQL-derived signals; score is not mislabelled as calibrated probability.

9.4 Combined discovery response test

1. Student submits a discovery request with skills and domain filters. Neo4j traverses shared skills, domain specialisation and optional department/company evidence.
2. Recommendation service combines candidates and evidence, then reads current MySQL eligibility, profile visibility, mentor active status, capacity and rating signals.
3. Redis may cache the final safe display response under a filter digest and recommendation epoch. The API still performs required authorization and current eligibility checks.
4. Update a mentor skill or deactivate an account. Commit the MySQL row and outbox event, let the worker update graph projection and invalidate the relevant cache, then repeat the same search.
5. Assert stale candidates are removed, explanations reflect current evidence, no private data is returned, and replaying the same event does not duplicate graph state or repeatedly bump cache generations.

10. Security, authorization and privacy verification

Security tests must be executed at the FastAPI and database integration boundary. Frontend route guards and hidden buttons are usability controls, not authorization. The actor ID and role must come from the authenticated server-side principal, not a request body, query parameter or cached object.

10.1 Security acceptance catalogue

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
SEC-01	P0	Unauthenticated caller requests protected Student profile.	GET protected route without credentials.	Rejected; no profile data returned.
SEC-02	P0	Student submits a different user_id in request body.	Override actor/user ID in payload.	Server ignores/rejects supplied identity; authenticated principal is used.
SEC-03	P0	Student reads another Student profile by ID.	Change profile ID in URL.	Denied/not-found; no private fields leaked.
SEC-04	P0	Student updates another Student profile.	PATCH foreign profile ID.	Denied; original profile unchanged.
SEC-05	P0	Student calls Mentor request-decision endpoint.	Invoke accept/reject directly.	403/denied; no status transition or allocation.
SEC-06	P0	Mentor A accepts request addressed to Mentor B.	Use Mentor A token and Mentor B request ID.	Denied; request stays PENDING.
SEC-07	P0	Mentor publishes availability for another mentor_id.	Change mentor_id in body.	Denied; no availability row created.
SEC-08	P0	Student books against another Student's allocation.	Change allocation ID.	Denied; no booking/slot/outbox write.
SEC-09	P0	Student reads another user's booking/session by ID.	Change object ID.	Denied/not-found; no participant data leakage.
SEC-10	P0	Mentor A records outcome for Mentor B's session.	Call outcome endpoint with foreign session.	Denied; session state/notes unchanged.
SEC-11	P0	Nonparticipant submits feedback.	Use unrelated principal.	Denied; no feedback or notification event.
SEC-12	P0	Mentor attempts to submit a student-authored mentor review.	Submit review as Mentor.	Denied; no review row.
SEC-13	P0	Student reviews CANCELLED/NO_SHOW session.	Submit review for invalid outcome.	Denied; no review inserted.
SEC-14	P0	Second mentor review for same completed session.	Repeat same session/student key.	Unique/business rule rejects duplicate; original preserved.
SEC-15	P0	Client sends role=ADMIN or is_admin=true.	Alter request body/token claims not signed by issuer.	No privilege change; server-side role remains authoritative.
SEC-16	P0	Administrator changes role or suspends account.	Use explicit permission.	Only permitted action succeeds; audit has actor/time/target/reason.
SEC-17	P0	Administrator attempts default impersonation, request acceptance or session outcome edit.	Invoke restricted capability.	Denied unless separately approved/audited capability exists.

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
SEC-18	P0	Public mentor card cache contains stale/private field.	Prime cache with intentionally invalid private field.	API returns approved public DTO only; privacy filtering still executes.
SEC-19	P0	Neo4j returns inactive mentor candidate.	Inject inactive candidate.	MySQL eligibility excludes from displayed/actionable results.
SEC-20	P1	Redis or Neo4j unavailable.	Stop each dependency in turn.	No authorization bypass; controlled fallback/degraded response.
SEC-21	P0	Suspended account attempts request/availability/booking/review.	Authenticate suspended user.	Write denied; historical rows remain readable under policy.
SEC-22	P0	Logs contain password, token, challenge secret or DB credential.	Inspect structured logs during auth/failure tests.	Test fails on any secret leakage.
SEC-23	P1	Worker receives malformed or unknown event.	Submit unsupported event type.	Event quarantined/rejected; no arbitrary write or unauthorized delivery.
SEC-24	P0	User requests another person's notifications.	Change user/inbox ID.	Denied; only own notification inbox returned.

10.2 Privacy assertions

- Public mentor directory and Redis public card contain only approved display fields such as display name, headline, company display name, experience, public skills/domains and completeness indicator.
- Public outputs do not expose login email, phone, internal notes, admin moderation fields, private biography, access token, password hash or participant-only session notes.
- Student can view own private profile and own workflow records. Mentor can view relevant student information for accepted allocations and sessions according to the approved disclosure model.
- Administrator can access permitted account/catalogue/report functions but does not automatically gain impersonation, session-outcome editing or authorship of a student review.
- Account deactivation is soft and does not erase historical requests, allocations, bookings, sessions, feedback or reviews.
- Administrative changes and sensitive exports are auditable where implemented; audit payload excludes credentials and unnecessary sensitive data.

11. Performance and capacity evaluation

PERFORMANCE TARGET PROVENANCE

The supplied PRD labels the following as an initial proposed baseline [SHOULD], not as a guaranteed blueprint-level SLA. The team should confirm targets against the actual deployment environment and test dataset before final sign-off.

WORKLOAD	INITIAL PROPOSED TARGET	MEASUREMENT RULE
Common read/API operations	p95 <= 2.5 seconds	Measure from API request receipt to response; record external network exclusions.
Mentor recommendations	p95 <= 4 seconds	Include Neo4j traversal, MySQL eligibility and required API response construction.
Booking confirmation	p95 <= 3 seconds	Measure the committed decision response; external notification delivery is asynchronous and excluded.
Concurrency profile	Up to 50 concurrent users	This is the initial PRD test profile; record workload mix and test environment.

11.1 Metrics to capture

LAYER	METRICS
API / application	p50, p95, p99 latency; request throughput; error rate; timeout rate; CPU/memory; connection-pool saturation.
MySQL / InnoDB	Query latency; rows examined; slow queries; lock wait time; deadlock count (1213); lock timeout count (1205); transaction duration; connection use.
Redis	Cache hit/miss by key family; TTL/expiry behaviour; evictions; memory; rejected writes; key cardinality; invalidation processing delay.
Neo4j	Traversal latency; rows/path count; query plans/PROFILE; index usage; failed queries; projection lag; stale-event rejection count.
Outbox / worker	Pending backlog; oldest pending age; processing duration; attempts; failed/dead-letter count; duplicate-suppression count; event-to-projection latency.
Functional correctness under load	Duplicate booking count; over-capacity allocation count; invalid state count; authorization leakage count; partial transaction count. Must remain zero.

11.2 Workload scenarios

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
PERF-01	P1	Baseline read workload.	Warm up; run mentor directory, profile, catalogue and dashboard reads at target concurrency.	Record p50/p95/p99, throughput, error rate and cache hit ratios; common reads p95 compared with proposed target.
PERF-02	P1	Recommendation workload.	Run skill/domain/department recommendations with representative graph size and filters.	p95 compared with proposed 4-second target; record graph latency and MySQL eligibility-query cost.

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
PERF-03	P0	Booking workload under mixed traffic.	Submit independent valid bookings across non-conflicting slots; separately run same-slot race.	p95 compared with proposed 3-second target; no duplicate slot or overlapping appointment.
PERF-04	P1	Cold-cache versus warm-cache comparison.	Run same query first with empty Redis then repeat warmed.	Compare response latency and MySQL query volume; correctness identical.
PERF-05	P1	Cache-expiry / invalidation burst.	Expire or invalidate popular profile/recommendation keys.	No stampede beyond expected load; rebuilds bounded; API stays correct.
PERF-06	P1	Outbox backlog recovery.	Pause worker, generate events, resume worker.	Backlog drains at measured rate; no duplicate/false notifications; no business write loss.
PERF-07	P1	Sustained / soak workload.	Run stable representative user mix for an agreed duration (recommended 30-60 minutes initially).	No progressive memory leak, connection exhaustion, backlog growth or correctness regression.
PERF-08	P2	Spike workload.	Increase concurrency above baseline briefly, then return to baseline.	System returns to steady state; failures are controlled and no invalid writes occur.

11.3 Recommended workload mix

For a repeatable initial benchmark, the team may begin with a balanced workload such as: 20% profile/catalogue reads; 20% mentor discovery/recommendations; 15% request lifecycle; 20% availability/booking; 10% session/feedback/history; 10% admin reports; 5% authentication/account operations. This mix is a suggested test profile, not a supplied requirement. Record the actual mix in every run.

11.4 Performance experiment procedure

1. Freeze the build SHA, schema migration version, MySQL isolation setting, Redis TTL/configuration, Neo4j version/index state and dataset size.
2. Run a warm-up phase; separate cold-cache and warm-cache measurements.
3. Ramp users gradually to the 50-user PRD baseline; record p50/p95/p99, throughput and error rate at steady state.
4. Capture application, MySQL, Redis, Neo4j and worker metrics over the same time window.
5. Repeat at least three benchmark runs if practical; report median and spread rather than one best result.
6. For any optimization, compare before/after against the same dataset and workload; retain EXPLAIN/PROFILE evidence.

12. Reliability, resilience and failure injection

12.1 Dependency failure tests

TEST ID	PRIORITY	SCENARIO / SETUP	EXECUTION	EXPECTED RESULT / EVIDENCE
RES-01	P0	MySQL unavailable during critical write.	Stop DB or reject connection.	API returns controlled failure; no partial state; retry is safe.
RES-02	P1	Redis unavailable during profile/cache read.	Stop Redis.	Fallback to MySQL where safe; response may be slower but correct.
RES-03	P1	Redis unavailable during temporary login challenge or rate-limit path.	Stop Redis during verification.	Fail safely; do not bypass verification or silently remove security check.
RES-04	P1	Neo4j unavailable during recommendation query.	Stop graph service.	Controlled degraded discovery (supported SQL fallback) or clear unavailable response; no invented eligibility.
RES-05	P1	Outbox worker unavailable.	Stop worker while commits continue.	Committed events stay pending; worker processes after recovery.
RES-06	P1	Notification provider repeatedly fails.	Return transient then permanent provider errors.	Retry with bounded backoff; failed event diagnosable/dead-lettered; booking/request remains committed.
RES-07	P1	Database restart after successful commit.	Commit known request/booking, controlled restart, query after recovery.	Committed durable state and outbox event remain.

12.2 Retry and error mapping

CONDITION	EXPECTED APPLICATION BEHAVIOUR	TEST ASSERTION
MySQL 1213 - deadlock	Rollback and retry the entire transaction/procedure with bounded retries and jitter.	No partial writes; second attempt uses fresh transaction; retries capped.
MySQL 1205 - lock wait timeout	Rollback full operation; return controlled retryable conflict and log contention.	No partial state; useful correlation ID and metrics.
MySQL 1062 - duplicate key	Interpret by constraint: slot already booked, allocation exists, review already submitted or idempotency replay.	Meaningful domain response; no generic 500 where known.
MySQL 45000 - business rule SIGNAL	Do not blindly retry; map to a validation/conflict response.	No duplicate attempts or misleading success response.
Redis/Neo4j/provider transient failure	Do not roll back a committed MySQL business transaction; keep durable event pending and retry.	Business row stays committed; event status observable.

12.3 Backup, restore and rebuild

- Perform a MySQL backup and restore into an isolated environment; compare critical row counts, FK integrity and key workflow queries.
- Rebuild Neo4j projection from authoritative MySQL snapshots/events, compare node/relationship counts and representative traversals, then flush only the isolated test Redis namespace and prove that profile/catalogue/recommendation values rebuild; rerun the end-to-end journey and same-slot race.

13. Reporting, query plans and operational observability

13.1 SQL/report validation

REPORT / QUERY	VALIDATION METHOD	ACCEPTANCE
Request-to-allocation-to-session funnel	Compare counts from known fixture IDs with SQL aggregates/CTEs.	Submitted, accepted, allocated, booked and completed counts reconcile to base rows.
Mentor rating summary	Recompute AVG(rating), COUNT(reviews) and completed sessions independently.	Only permitted completed-session reviews included; rounding documented.
Pending requests by mentor	Compare view rows with direct status filter.	Only addressed Mentor's pending requests appear.
Mentor booking calendar	Compare view with confirmed future bookings.	No cancelled slot is shown as confirmed; access scope correct.
Students without allocation	Compare anti-join/NOT EXISTS result with fixture.	No false inclusion/exclusion; duplicate request rows do not multiply student result.
Mentor domain ranking	Validate grouping/window rank by domain.	Rank is deterministic for ties using declared tie-breaker.

13.2 Query-plan evaluation

- Capture EXPLAIN for mentor discovery, booking calendar, request inbox, student mentorship history and rating summary.
- Use EXPLAIN ANALYZE where supported in the configured MySQL version and safe test environment; compare actual rows/time.
- For Neo4j traversal, use EXPLAIN/PROFILE in a controlled dataset; verify expected uniqueness indexes and bounded traversal shape.
- Compare before/after a relevant index or query rewrite using the same dataset, parameter values and cache condition.
- Document plan shape, rows examined, execution time, index access and any trade-off; do not claim optimization from one timing run.

13.3 Operational dashboard / log checks

SIGNAL	WHAT TO REVIEW	EXPECTED CONTROL
Request correlation	Request ID, actor ID, module, operation, outcome and duration.	Can trace a request across API, SQL transaction and worker event.
Sensitive logging	Password, OTP/challenge, token, credential, DB URL and personal data.	Redaction/allow-list prevents secrets in logs.
Transaction health	Commit/rollback counts, 1213, 1205, lock wait, duration.	Failures visible and classified; no unexplained retries.
Cache health	Hit/miss/eviction, memory, invalidation delay.	Cache state disposable; high misses or stale values diagnosable.
Graph health	Projection lag, event version, retries, rebuild time.	Stale projection visible; MySQL eligibility still authoritative.
Outbox health	Pending count, oldest event age, attempts, dead-letter status.	No silent drop; retry state observable.

14. Execution process, defect management and exit gates

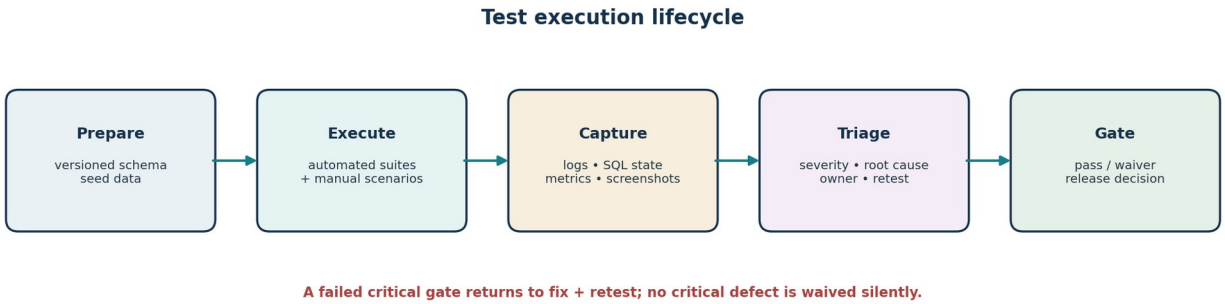


Figure 3. Test execution and release-decision flow.

14.1 Entry criteria

- Approved conceptual ER model and physical schema mapping are available; migrations can create the test database from empty state.
- Business rules and open policy decisions that affect test outcomes are recorded (capacity, cancellation cutoff, session outcome authority, review visibility/edit and role combinations).
- FastAPI routes and authentication dependencies are available; MySQL/Redis/Neo4j test instances are isolated.
- Outbox schema, event types, worker status transitions and event-idempotency approach are documented.
- Synthetic fixtures and test secrets are managed outside source control; test data reset is repeatable.

14.2 Execution order

PHASE	ACTIVITIES	EXIT CHECK
1. Foundation	Schema migrations, identity, roles, profile setup and DBI tests.	PK/FK/unique/check and role tests pass.
2. Catalogue/profile	Skills, domains, departments, companies and public/private projections.	M:N relations, referential integrity and public DTO tests pass.
3. Requests/allocation	Submit, accept, reject, cancellation, capacity and audit.	WF request tests and TX-01/TX-02 pass.
4. Availability/booking	Slot publication, booking/cancellation, concurrency and idempotency.	TX-03..TX-09 and workflow booking tests pass.
5. Sessions/feedback	Outcome, participant feedback, mentor review and history.	Only valid participants/outcomes; duplicates rejected.
6. Notifications/admin	Outbox worker, recipients, retries, audit, admin reports.	OUT suite, FI tests and report reconciliations pass.
7. Redis	Cache-aside, TTL, invalidation, rate limits and idempotency helpers.	RED-01..RED-16 pass; Redis-off correctness verified.
8. Neo4j	Graph projection, traversal and explainable recommendations.	G-01..G-12 pass; MySQL eligibility filter verified.
9. NFR / acceptance	Security regression, load tests, resilience, UX and demo.	Exit criteria met; evidence pack archived.

14.3 Defect severity and handling

SEVERITY	EXAMPLES	RELEASE HANDLING
Critical / Blocker	Unauthorized data exposure; double booking; over-capacity allocation; committed write lost; false notification for rolled-back action; partial transaction.	Stop affected release; fix and rerun full regression for impacted workflow and dependencies.

SEVERITY	EXAMPLES	RELEASE HANDLING
High	Outbox/graph/cache retry failure; wrong public/private projection; major workflow blocked; admin data incorrect.	Fix before release of affected capability; rerun integration and regression.
Medium	Non-critical report mismatch with workaround, UI issue, ranking tie-break defect not affecting eligibility.	Document owner/workaround; confirm no data-integrity/security effect; schedule fix.
Low	Cosmetic/spacing/copy issue.	Track; does not block correctness/security gates.

14.4 Exit criteria

MANDATORY EXIT GATE

All P0 tests pass; all critical business rules are covered; no unresolved critical/high defect in identity, authorization, request/allocation, booking, session feedback, outbox or cross-store consistency; concurrency tests prove at most one booking per slot and no participant overlap; performance results are recorded against the agreed targets; all failures have an owner and disposition.

- Every test result is Pass, Fail, Blocked or Not Applicable with a reason; no blank status.
- All P0 and release-scope P1 test cases have execution evidence and retest result.
- Every waiver states risk, business impact, compensating control, owner and approval; no silent waiver.
- No unresolved data corruption, unauthorized access, duplicate allocation/booking, lost outbox intent or false-notification defect.
- Performance report includes test build, environment, exact dataset size, workload mix, concurrency, warm-up, run duration and raw metrics.
- Final evidence pack contains test summary, defects, schema/constraint outputs, concurrency transcript, cache/graph/outbox evidence and release decision.

15. Acceptance demonstration and evidence pack

15.1 Suggested complete demonstration

1. Student signs in, updates skills and interests, discovers mentors and sees evidence explaining why a candidate is relevant.
2. Student submits a request; Mentor accepts; one allocation is created in the same MySQL transaction and an outbox event is visible.
3. Mentor publishes a valid slot; two independent clients attempt the same slot concurrently; exactly one booking commits.
4. Mentor records an offline session as COMPLETED with topics, skills assessed, observations, improvement areas and next steps.
5. Actual participants submit session feedback; the Student submits one mentor review for the completed session; duplicate and nonparticipant attempts are rejected.
6. Administrator opens request-to-allocation-to-session funnel and mentor-rating reports; counts reconcile to SQL data.
7. Demonstrate Redis miss/hit/expiry and post-commit invalidation; show a stale availability preview cannot confirm a booking.
8. Demonstrate Neo4j shared-skill/domain traversal, recommendation explanation, MySQL capacity filter, event replay and projection rebuild.
9. Turn off Redis or Neo4j and show that committed MySQL state remains correct; restart worker and show pending events recover.

15.2 Evidence pack checklist

EVIDENCE ID	ARTIFACT	WHAT IT PROVES
EV-01	DDL/migration output; SHOW CREATE TABLE; indexes.	PK, FK, UNIQUE, CHECK, status domains and required indexes exist.
EV-02	SHOW TRIGGERS and procedure source/CALL output.	Invalid transitions rejected; audit and workflow routines behave.
EV-03	Before/after SQL snapshots for rollback injection.	No partial booking/allocation/session/outbox writes.
EV-04	Two-session concurrency transcript with connection IDs/barrier.	Same slot and overlapping participant races are safe.
EV-05	Security API test report and role matrix.	Positive and negative role/ownership checks at backend.
EV-06	Public mentor DTO/cache payload and log scan.	No private data or secrets exposed.
EV-07	Redis CLI / API hit-miss-expiry / invalidation logs.	Redis structures, TTL and post-commit invalidation work.
EV-08	Neo4j Browser/Cypher result, event replay and rebuild counts.	Graph identity, traversal, idempotency and recovery.
EV-09	Outbox pending/processed/failed status and worker logs.	Retries are safe and failed events are diagnosable.
EV-10	Load-test report and EXPLAIN/PROFILE artifacts.	Performance measured with known environment and dataset.
EV-11	End-to-end screenshots/API traces.	Student, Mentor and Admin journeys complete correctly.
EV-12	Defect register and release sign-off.	Known risks and acceptance decision are explicit.

16. Open decisions, assumptions and source traceability

16.1 Team decisions to finalize before test execution

DECISION	WHY IT AFFECTS TESTS	TEST-PLAN HANDLING UNTIL DECIDED
Maximum active mentees per Mentor	Acceptance race and capacity tests require exact limit.	Use a configurable fixture value; test below/at/above capacity.
Student active allocation policy	May restrict request/booking eligibility.	Record approved rule; do not invent one in tests.
Duplicate request definition	Determines uniqueness and expected repeated-submit behaviour.	Confirm unique key scope (e.g., student+mentor or student+mentor+domain).
Booking cancellation cutoff and slot release	Affects cancel flow, overlap and slot reopening.	Use test configuration; assert history retained and release policy explicit.
Booking/session creation timing	ER model says Booking -> 0..1 Session; workflow notes may create scheduled session at booking.	Choose one consistent policy and align all tests/DB constraints.
Session no-show/cancel authority	Affects authorization and session state tests.	Only permitted actor/role can mark; any admin correction requires reason/audit.
Feedback visibility/edit policy	Affects participant read paths and review revision tests.	Keep per-field policy in a testable access matrix.
One User may hold Student and Mentor roles?	Affects profile cardinality, active role context and route guards.	Test only approved role combination; graph design supports shared User node with multiple profiles.
Authentication/session/MFA/CSRF details	Affects auth, revocation, cookie/session, OTP and browser security tests.	Document chosen scheme; run checks appropriate to that scheme.
Rate limits, lockout, idempotency retention	Affects short-lived Redis state and abuse controls.	Values are configuration; no unapproved fixed threshold in test assertions.
Outbox retry/dead-letter settings	Affects worker backoff, delivery and alerting.	Confirm retry count, lease timeout, dedupe retention and operational owner.
Performance test resource baseline	Needed to interpret latency results fairly.	Record CPU, RAM, DB sizing, service versions and dataset.

16.2 Assumptions and interpretation

- MySQL 8.x with InnoDB is the authoritative relational database. The exact patch release and runtime configuration must be recorded when testing.
- The source documents state that MySQL is authoritative and Redis/Neo4j are supporting stores. Redis and Neo4j may be unavailable, stale or rebuilt without corrupting committed business state.
- The 50-concurrent-user profile and p95 response thresholds are proposed initial PRD baselines, not contractual guarantees or measured results.
- The exact TTLs, retry counts, lockout thresholds, cancellation windows, mentor capacity and review-edit policy remain configurable team decisions unless separately approved.
- Code snippets/table/column examples in prior specifications are templates; final tests must target the project's migrated schema and implemented endpoint contract.
- Optional Redis Stack vector search is Phase 2 and is not a blocker for core acceptance.

16.3 Source documents consolidated

SOURCE DOCUMENT	SECTIONS / PAGES USED IN THIS PLAN
Alumni–Student Mentorship Platform: DBMS Project Blueprint	Project scope and roles; modular monolith and polyglot persistence; MySQL/Redis/Neo4j ownership; workflows; DBMS Unit 1-4 mapping.
AlumniConnect Product Requirements Document	FR-19/FR-20 notifications and outbox; FR-23 Redis cache; BR-01..BR-12 data rules; NFR-01..NFR-10 security, integrity, consistency, performance, reliability, audit and usability.

SOURCE DOCUMENT	SECTIONS / PAGES USED IN THIS PLAN
AlumniConnect Application Architecture	Sections 7-9 Redis/Neo4j, outbox and booking concurrency; Section 13 security/reliability; Section 15 implementation sequence and acceptance checklist.
AlumniConnect Workflow and State Diagrams	Request/booking/session lifecycles; Unit 4 ACID and concurrency; Section 19 Verification Plan WF-01..WF-16; build order and open decisions.
AlumniConnect Business Rules Catalogue	Integrity and status rules; Section 12 transactions/isolation/deadlocks; Section 15 privacy/retention; Section 16 verification BR-T01..BR-T34; policy decisions.
AlumniConnect Conceptual ER Diagram & Domain Model	Conceptual entity/cardinality baseline; 24 entities and 36 relationships; identity, skills, request/allocation, scheduling, feedback and operations domains.
AlumniConnect Use Case Model	Administrator authority; UC-S09 booking; session/feedback/review; end-to-end scenarios; data-to-database mapping; core versus optional scope.
AlumniConnect Redis Design Specification	Key naming and data structures; TTL/invalidation; cache-aside; outbox integration; Section 18 RED-01..RED-16 acceptance.
AlumniConnect Neo4j Graph Design	Graph identity and traversal; MySQL-to-Neo4j synchronization; Section 17 G-01..G-12 test and acceptance catalogue.
AlumniConnect Transaction and Consistency Design	ACID boundaries; locking and isolation; Section 22 TX-01..TX-14; Section 23 failure injection/rollback evidence.
AlumniConnect Security and Authorization Design	RBAC plus ownership; public/private boundaries; SEC-01..SEC-24 negative/positive authorization tests; database/worker privilege boundaries.
Supplied DBMS Unit 2 and Unit 4 course materials	Triggers, stored procedures, ACID, concurrency, lock-based protocols and deadlock concepts demonstrated by project workflows.

FINAL ACCEPTANCE PRINCIPLE

Do not accept a feature solely because the frontend displays a success message. Verify the authoritative MySQL state, the associated audit/outbox event, the permission boundary and any Redis/Neo4j side effect. When secondary stores disagree or fail, MySQL wins; the platform may become slower or temporarily degraded, but it must not become incorrect.